
Documentação de Projeto

para o sistema

FitCore - Sistema de Gestão de Academia

Versão 1.0

Projeto de sistema elaborado pelo aluno Arthur Modesto Couto
como parte da disciplina **Projeto de Software**.

<10/06/2026>

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema FitCore. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

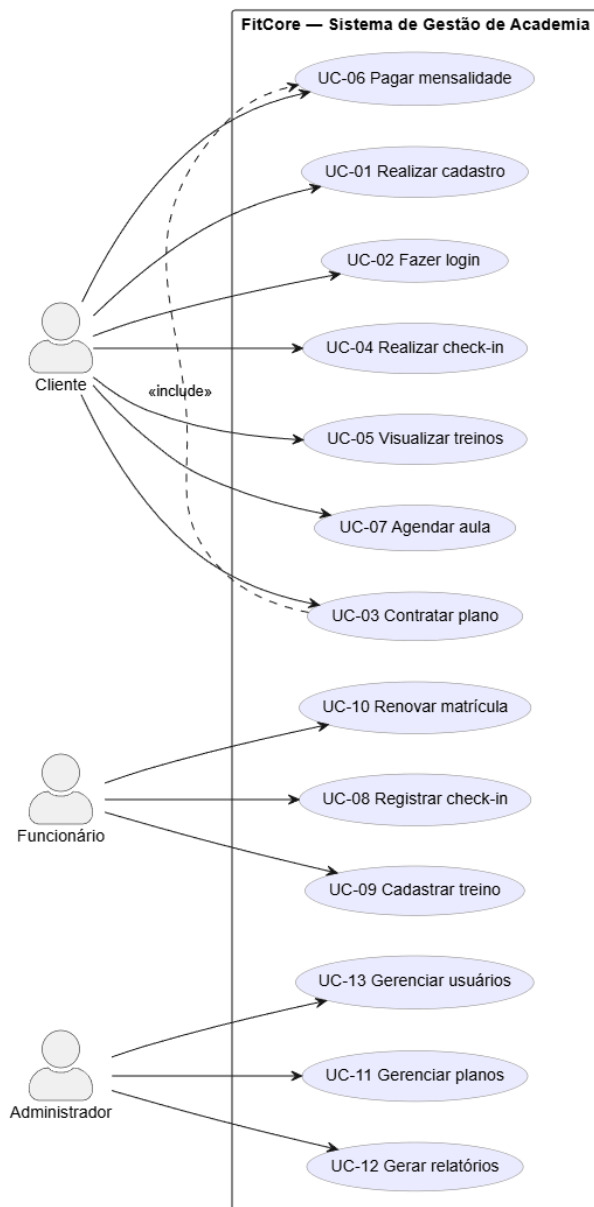
Nesta subseção é apresentada a descrição de cada um dos atores que interagem com o sistema FitCore. Um ator representa um papel desempenhado por uma pessoa, organização ou sistema externo que interage diretamente com o sistema.

Atores	Descrição
Cliente	Aluno matriculado na academia. Pode realizar seu próprio cadastro, contratar e renovar planos, efetuar pagamentos, realizar check-in, visualizar seus treinos e agendar aulas coletivas.
Funcionário	Colaborador da academia responsável pelo atendimento presencial. Registra check-ins de alunos na recepção, cadastra fichas de treino e realiza renovações de matrícula.
Administrador	Gestor do sistema com acesso total. Responsável por configurar planos disponíveis, gerenciar cadastros de usuários e funcionários, e acompanhar relatórios gerenciais da academia.

2.2 Modelo de Casos de Uso

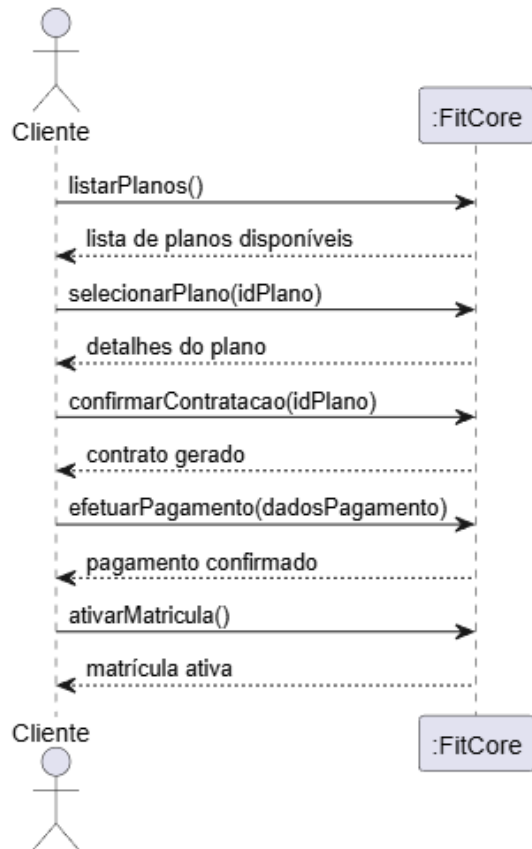
Nesta subseção é apresentado o diagrama de casos de uso do sistema FitCore. O diagrama ilustra as interações entre os três atores — Cliente, Funcionário e Administrador — e as funcionalidades disponíveis no sistema. Cada caso de uso recebe um identificador único no formato UC-XX, utilizado como referência ao longo deste documento. O relacionamento entre UC-03 e UC-06 indica que a contratação de um plano sempre envolve, obrigatoriamente, o pagamento de

mensalidade.



2.3 Diagrama de Sequência do Sistema

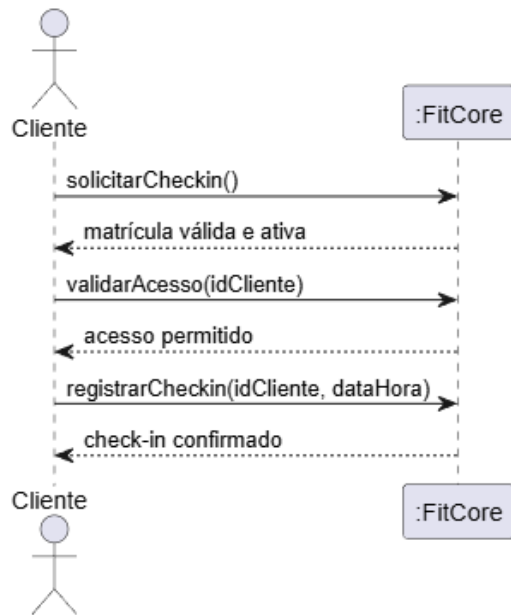
Nesta subseção são apresentados os Diagramas de Sequência do Sistema (DSS) para três casos de uso do FitCore: UC-03 Contratar Plano, UC-04 Realizar Check-in e UC-07 Agendar Aula. Os DSS ilustram a sequência de mensagens trocadas entre o ator e o sistema como uma caixa-preta, sem expor detalhes internos de implementação. Para cada diagrama é apresentado o respectivo contrato de operação.



Contrato	CO-01
Operação	listarPlanos()
Referências cruzadas	UC-03
Pré-condições	Cliente autenticado no sistema
Pós-condições	Lista de planos ativos retornada ao cliente

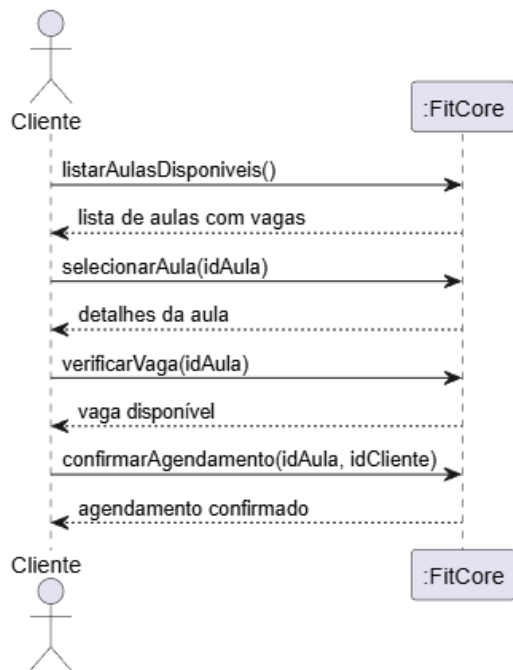
Contrato	CO-02
Operação	confirmarContratacao(idPlano)
Referências cruzadas	UC-03, UC-06
Pré-condições	Plano selecionado existe e está ativo; cliente sem plano vigente
Pós-condições	Contrato criado e associado ao cliente; status da matrícula alterado para "aguardando pagamento"

Contrato	CO-03
Operação	efetuarPagamento(dadosPagamento)
Referências cruzadas	UC-06
Pré-condições	Contrato gerado; dados de pagamento válidos
Pós-condições	Pagamento registrado; matrícula ativada; data de vencimento definida



Contrato	CO-04
Operação	<code>validarAcesso(idCliente)</code>
Referências cruzadas	UC-04
Pré-condições	Cliente autenticado; matrícula existente no sistema
Pós-condições	Verificado que a matrícula está ativa e não vencida

Contrato	CO-05
Operação	<code>registrarCheckin(idCliente, dataHora)</code>
Referências cruzadas	UC-04
Pré-condições	Acesso validado; cliente não realizou check-in no mesmo dia
Pós-condições	Registro de presença criado com data e hora; frequência do aluno atualizada



Contrato	CO-06
Operação	verificarVaga(idAula)
Referências cruzadas	UC-07
Pré-condições	Aula selecionada existe e está agendada para data futura
Pós-condições	Confirmado que há vagas disponíveis na aula

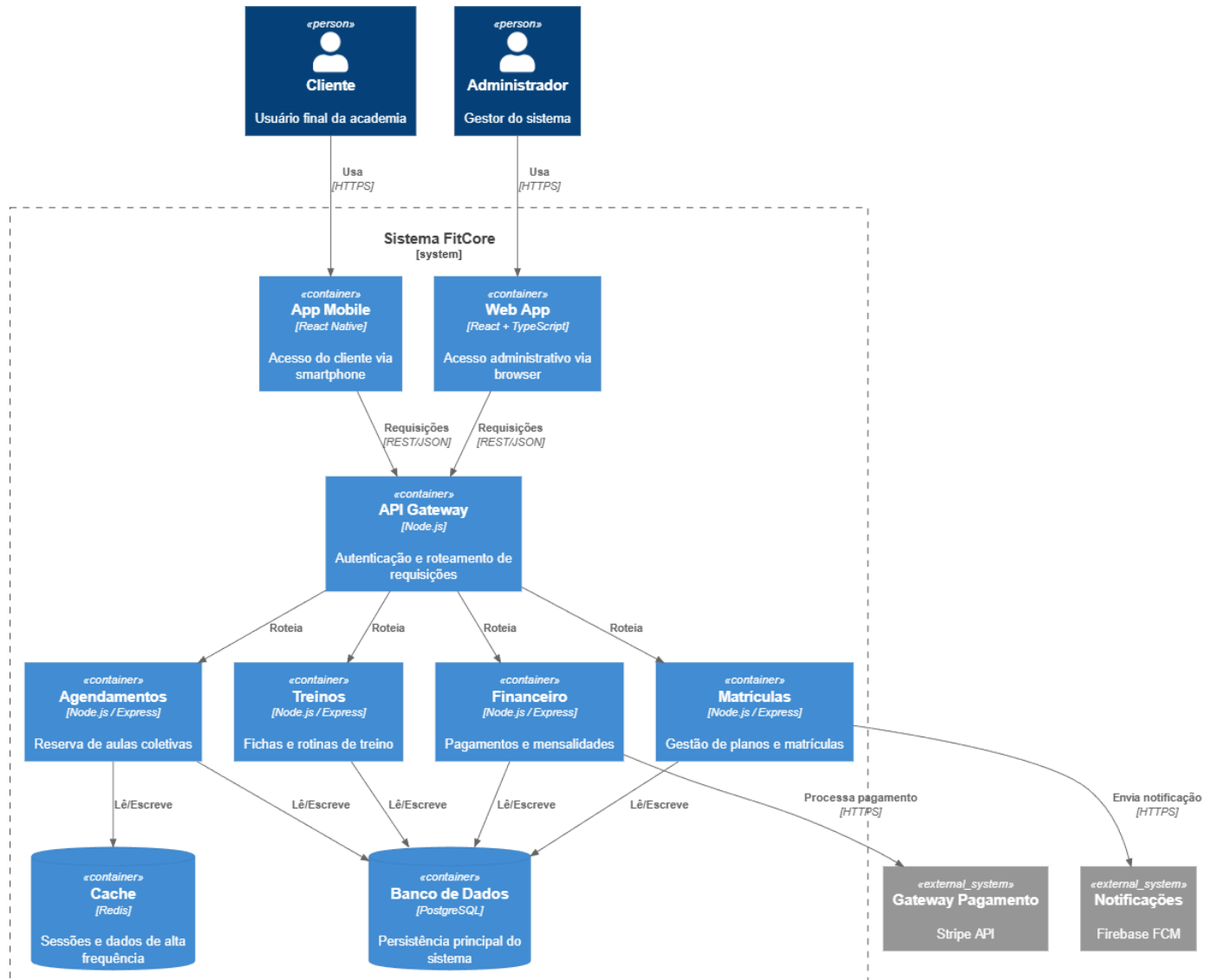
Contrato	CO-07
Operação	confirmarAgendamento(idAula, idCliente)
Referências cruzadas	UC-07
Pré-condições	Vaga disponível confirmada; cliente sem agendamento duplicado na mesma aula
Pós-condições	Reserva criada e associada ao cliente; contador de vagas da aula decrementado em 1

3. Modelos de Projeto

3.1 Arquitetura

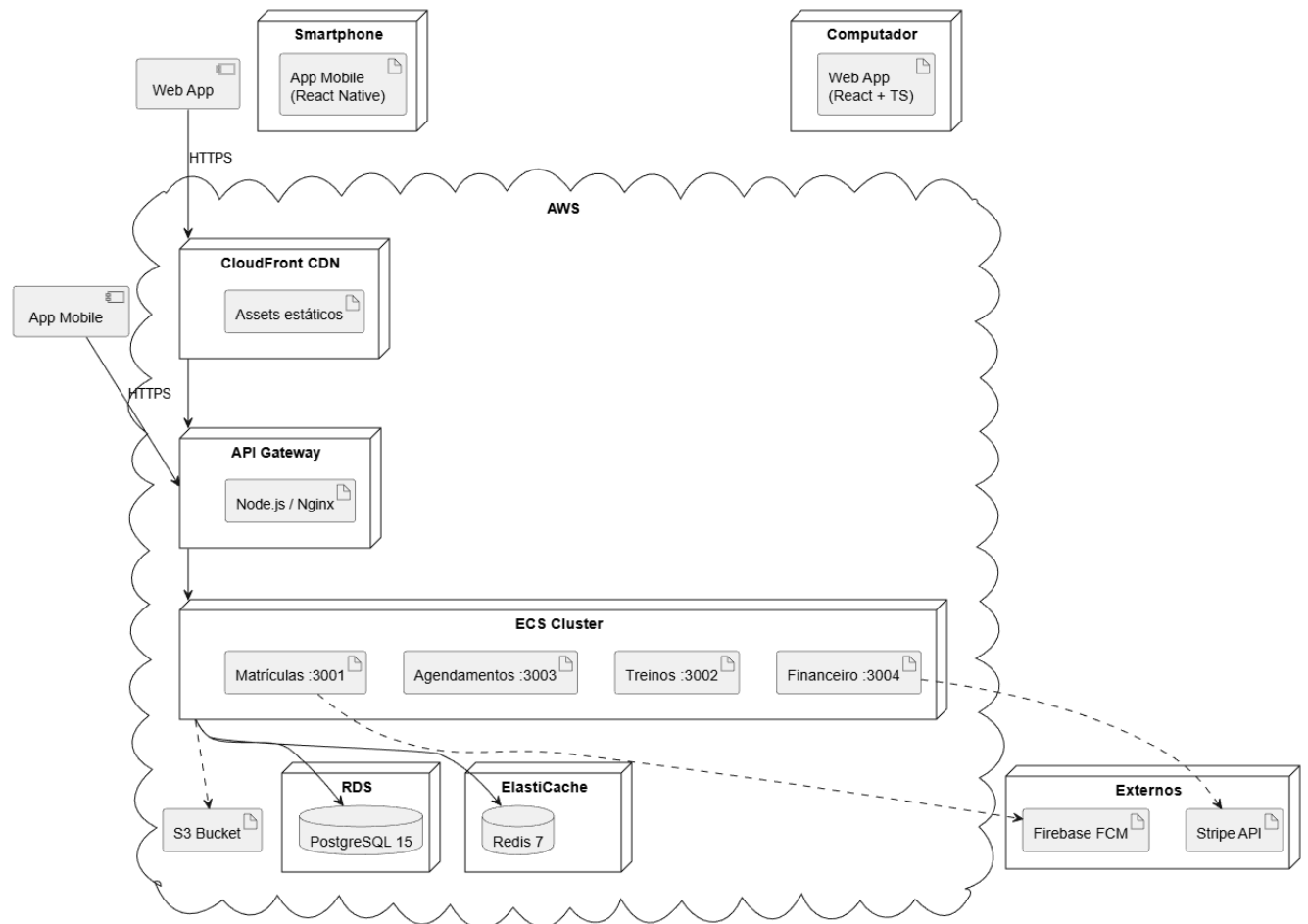
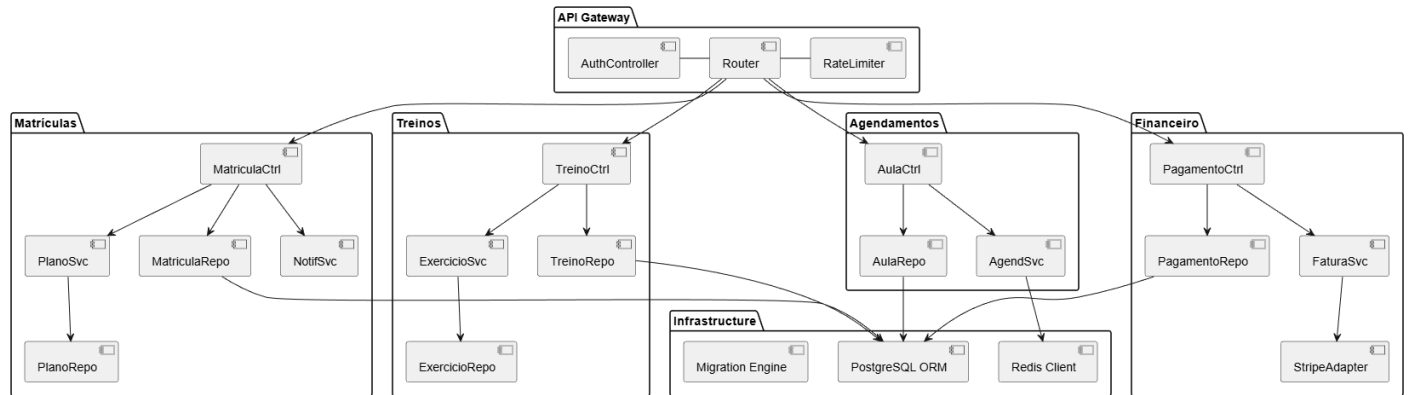
Nesta subseção é apresentada a arquitetura do sistema FitCore. A descrição segue o modelo C4, representando o nível de Contêineres — ou seja, as principais unidades do sistema e como elas se comunicam. A arquitetura adota o padrão em camadas com separação entre frontend, backend e persistência,

viabilizando escalabilidade e manutenibilidade.



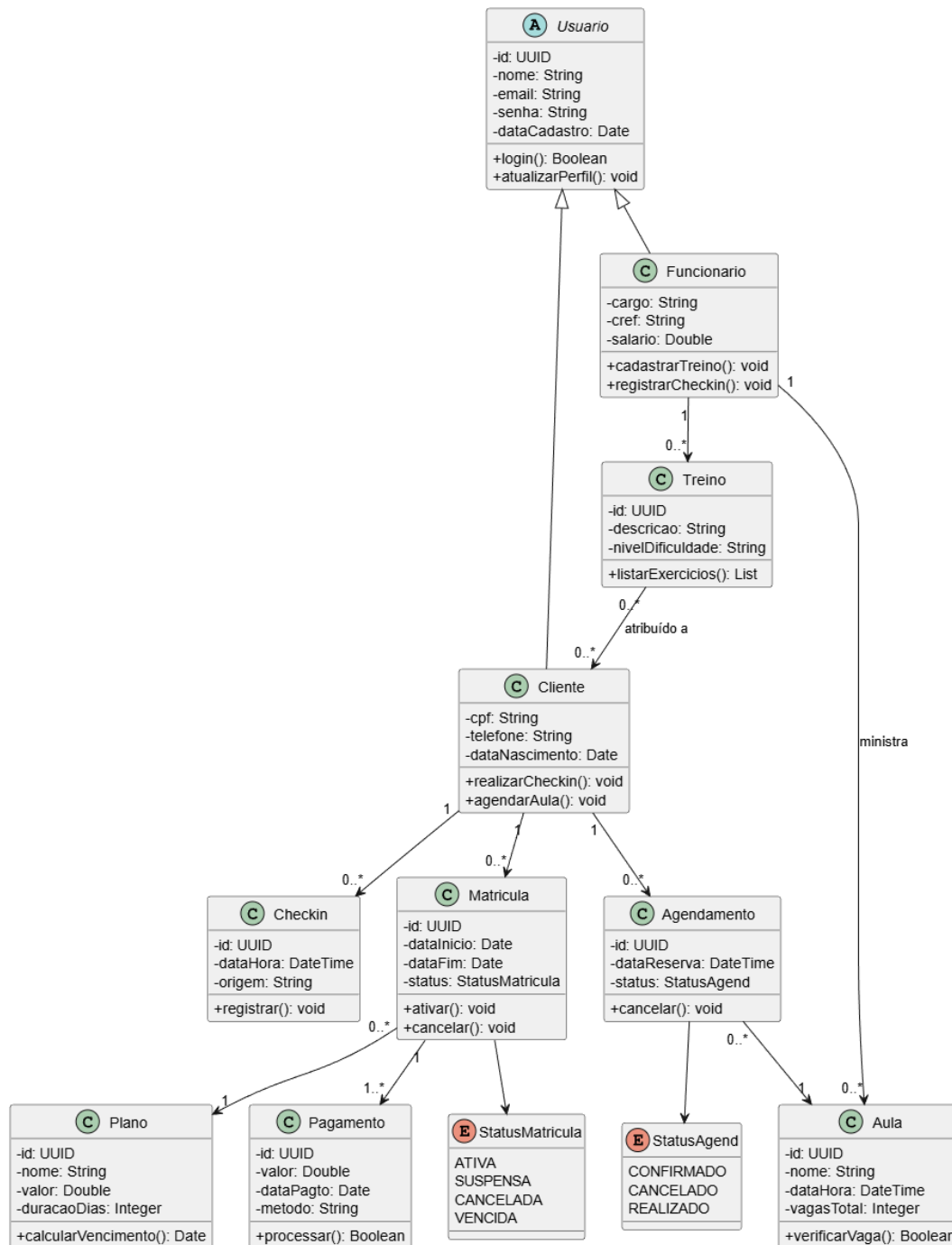
3.2 Diagrama de Componentes e Implantação.

Nesta subseção são apresentados o Diagrama de Componentes e o Diagrama de Implantação, respectivamente, do sistema FitCore. O diagrama de componentes detalha a organização interna dos módulos do backend, evidenciando as interfaces e dependências entre eles. O diagrama de implantação mostra como esses componentes são distribuídos fisicamente nos nós de execução em ambiente de produção.



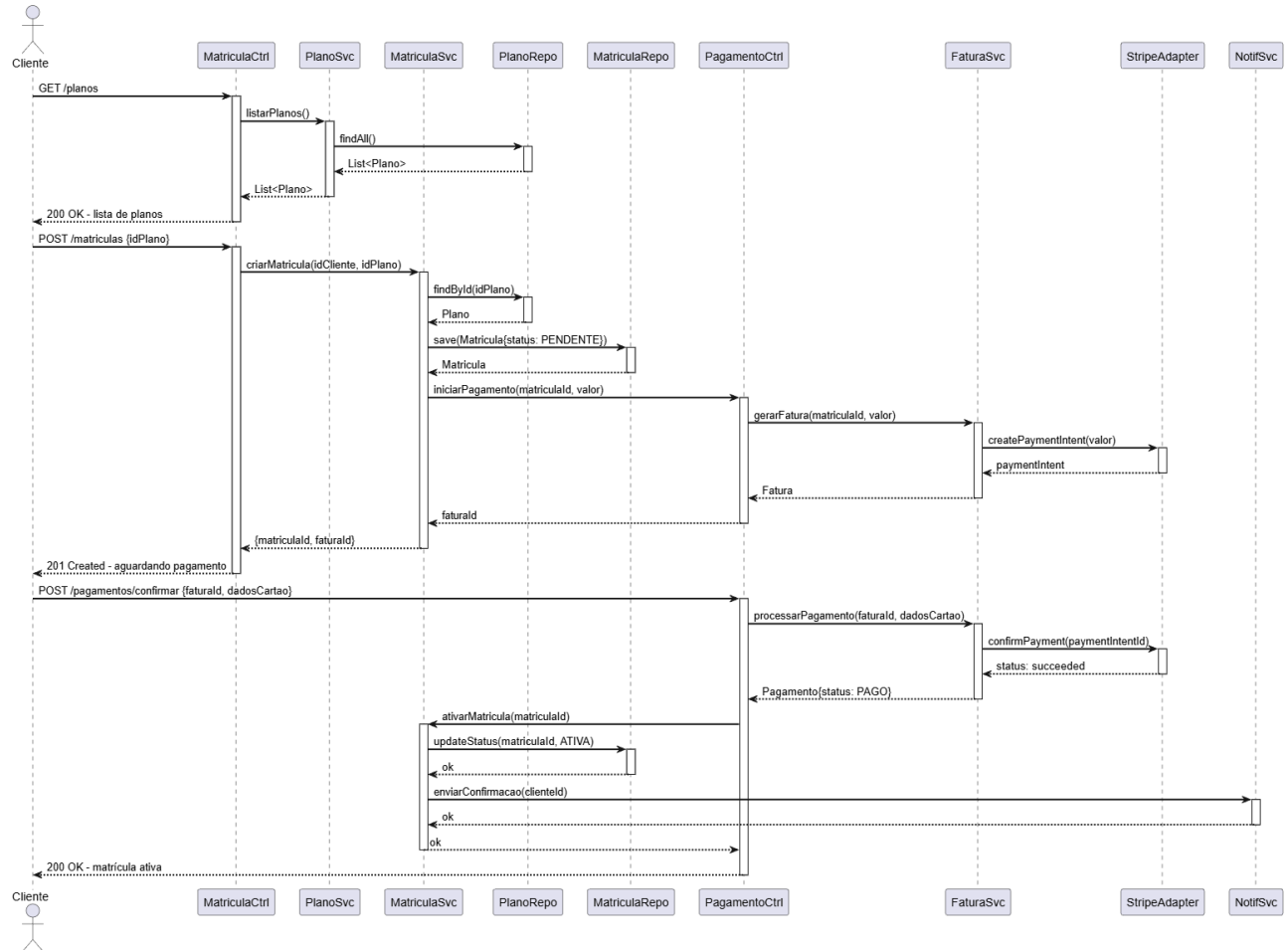
3.3 Diagrama de Classes

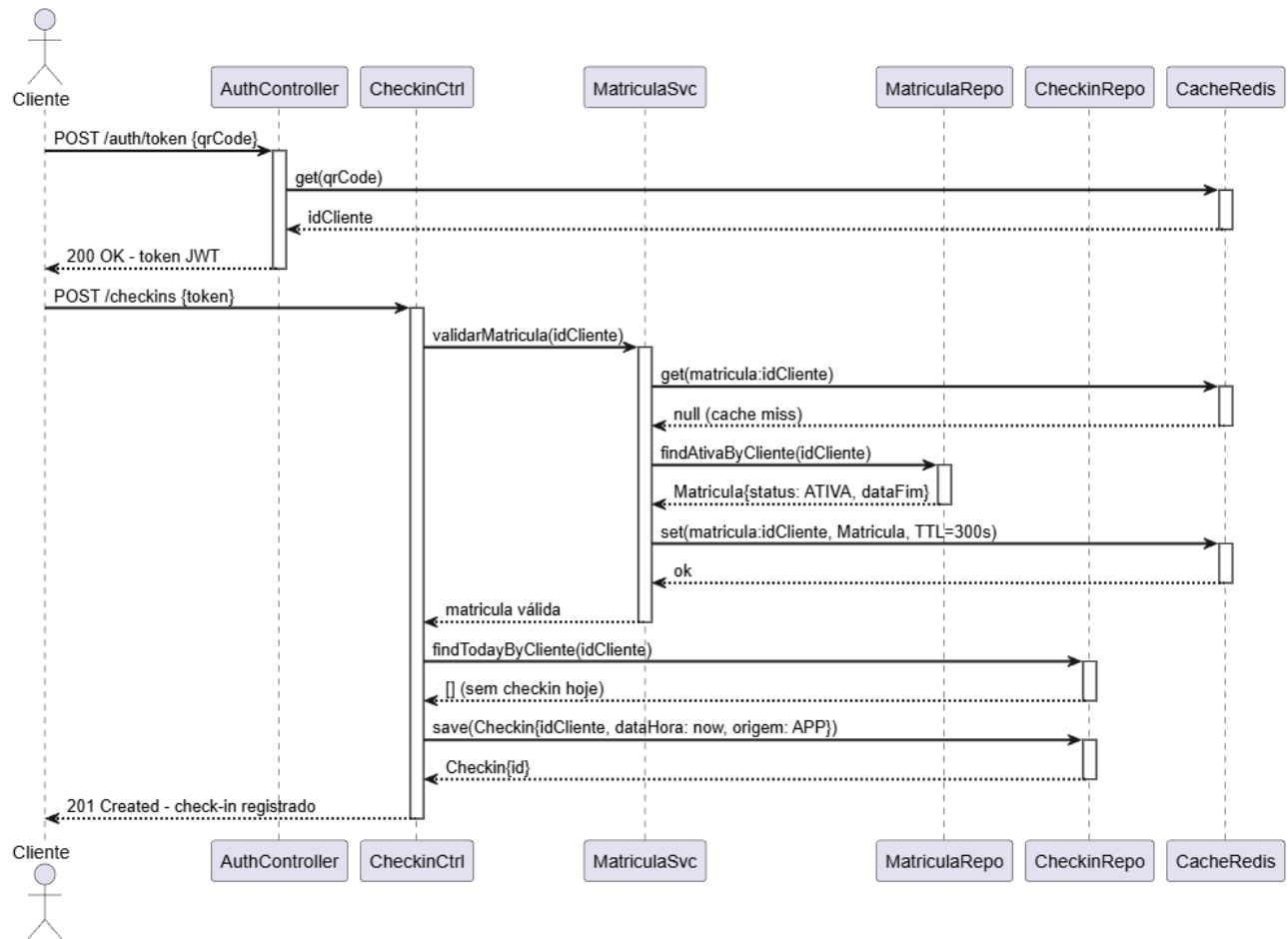
Nesta subseção é apresentado o Diagrama de Classes do sistema FitCore. O diagrama modela as principais entidades do domínio, seus atributos, métodos e os relacionamentos entre elas, servindo de base tanto para a implementação do backend quanto para o mapeamento objeto-relacional.

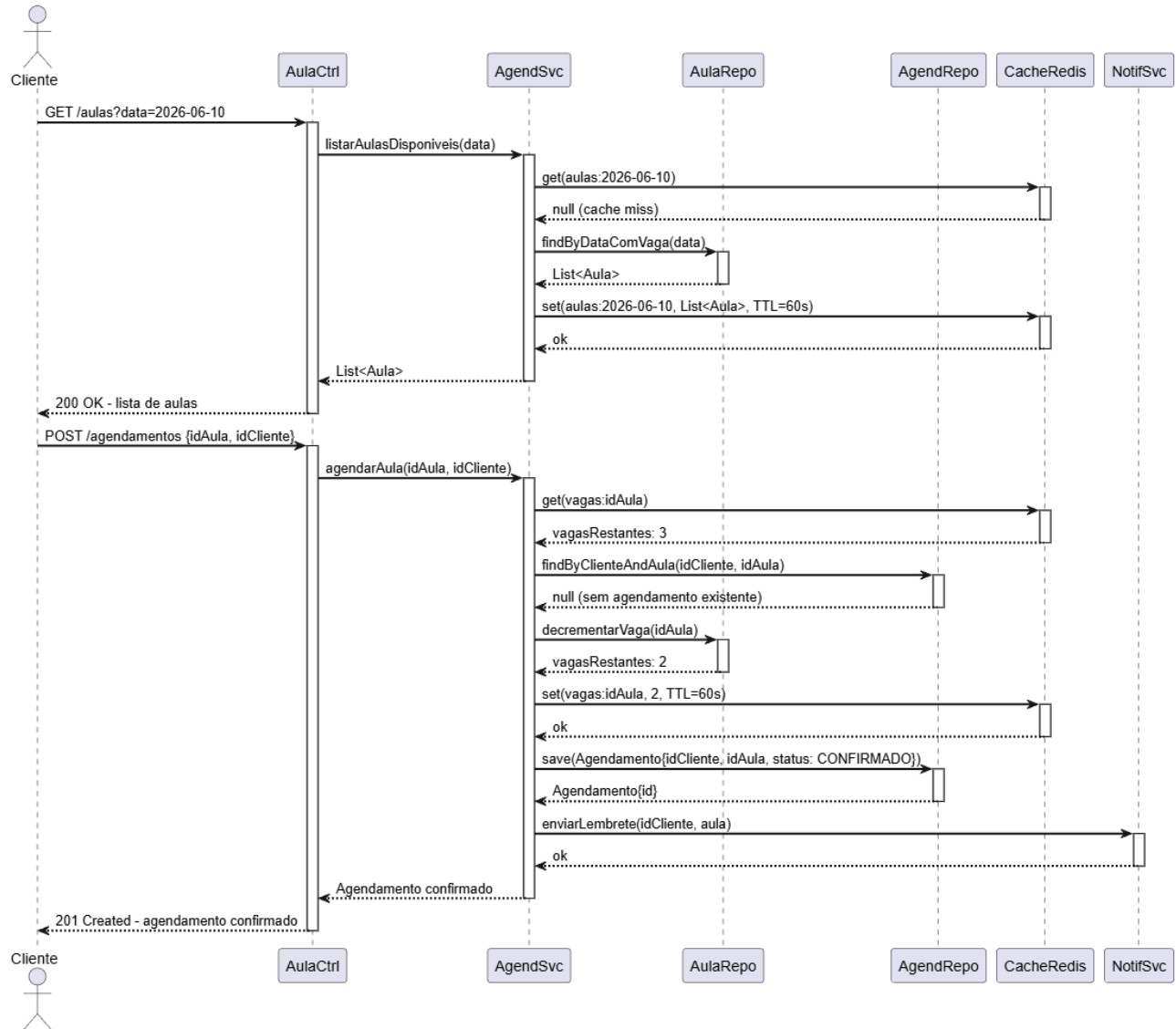


3.4 Diagramas de Sequência

Nesta subseção são apresentados os Diagramas de Sequência de Projeto para os casos de uso UC-03, UC-04 e UC-07. Diferentemente dos diagramas da seção 2.3, aqui os objetos internos do sistema são expostos, evidenciando a colaboração entre controllers, services, repositories e entidades de domínio durante a execução de cada operação.

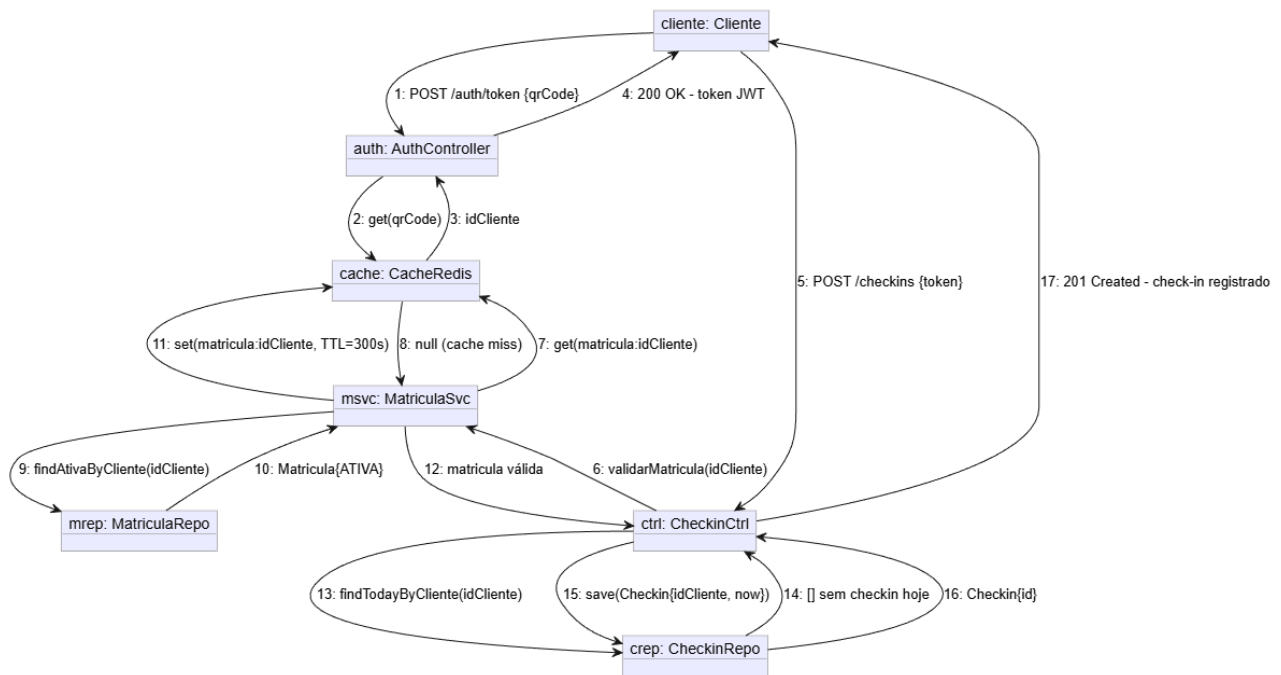
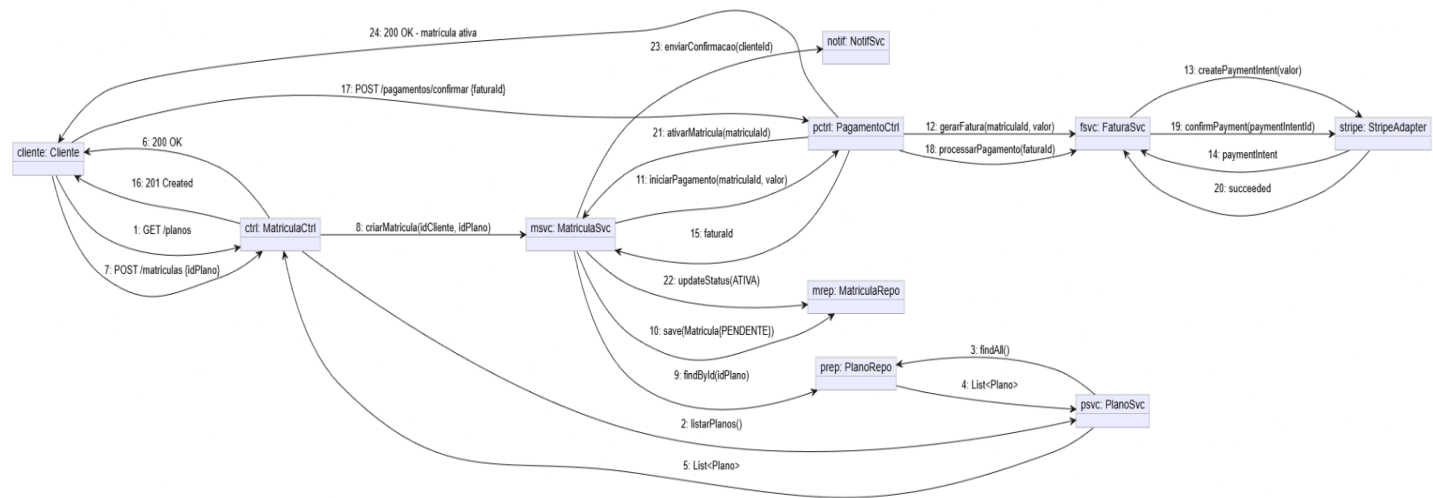


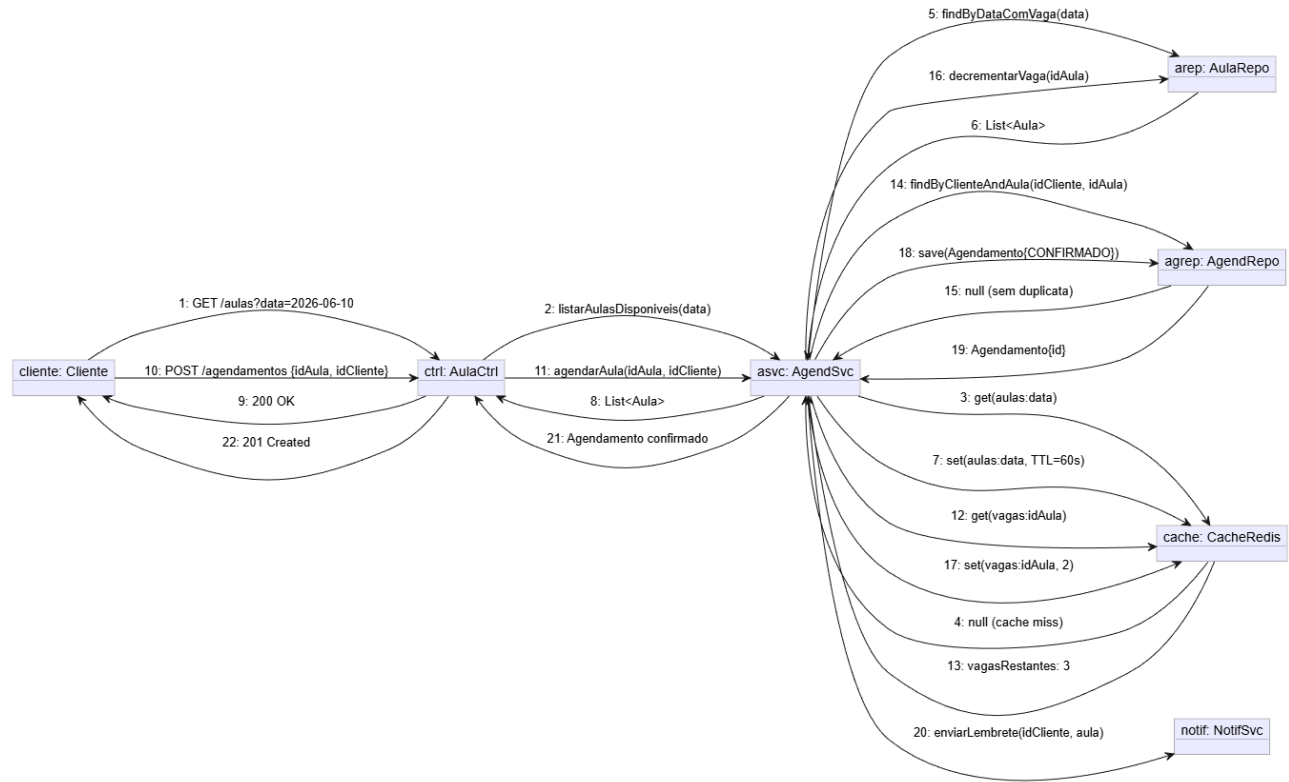




3.5 Diagramas de Comunicação

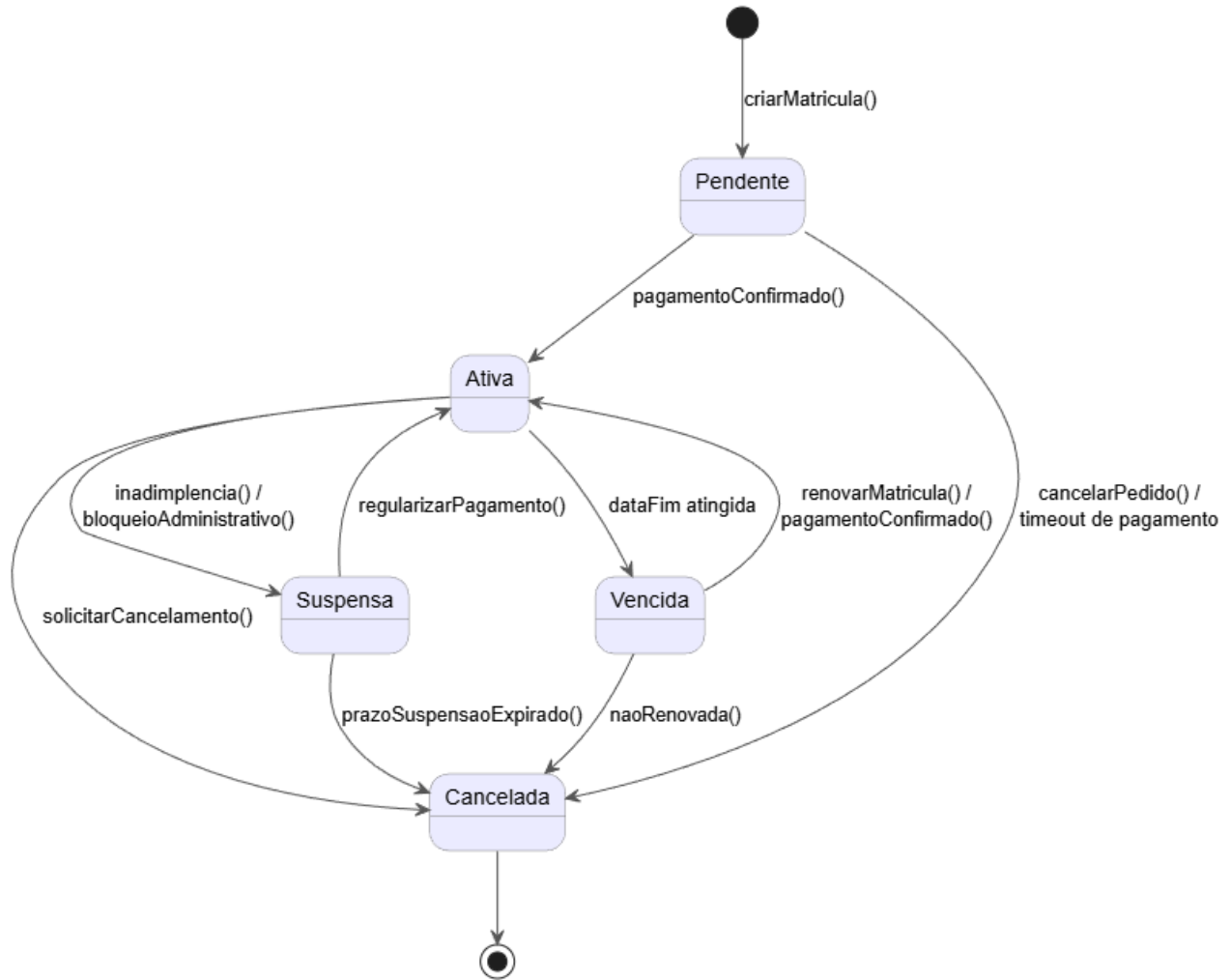
Nesta subseção são apresentados os Diagramas de Comunicação para os casos de uso UC-03, UC-04 e UC-07. Diferentemente dos diagramas de sequência, aqui o foco recai sobre os vínculos estruturais entre os objetos e a numeração das mensagens trocadas, evidenciando como cada participante colabora para a realização do caso de uso.

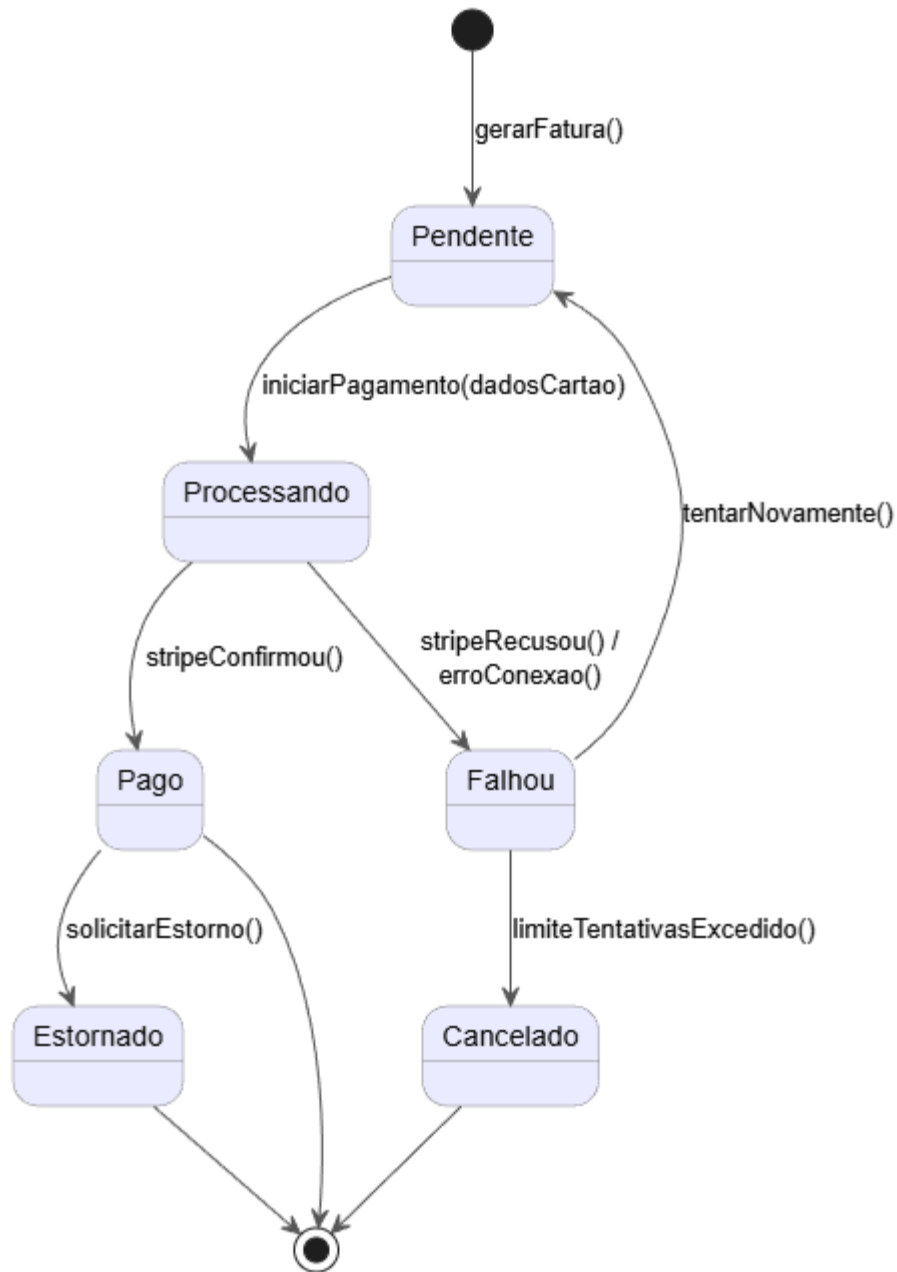


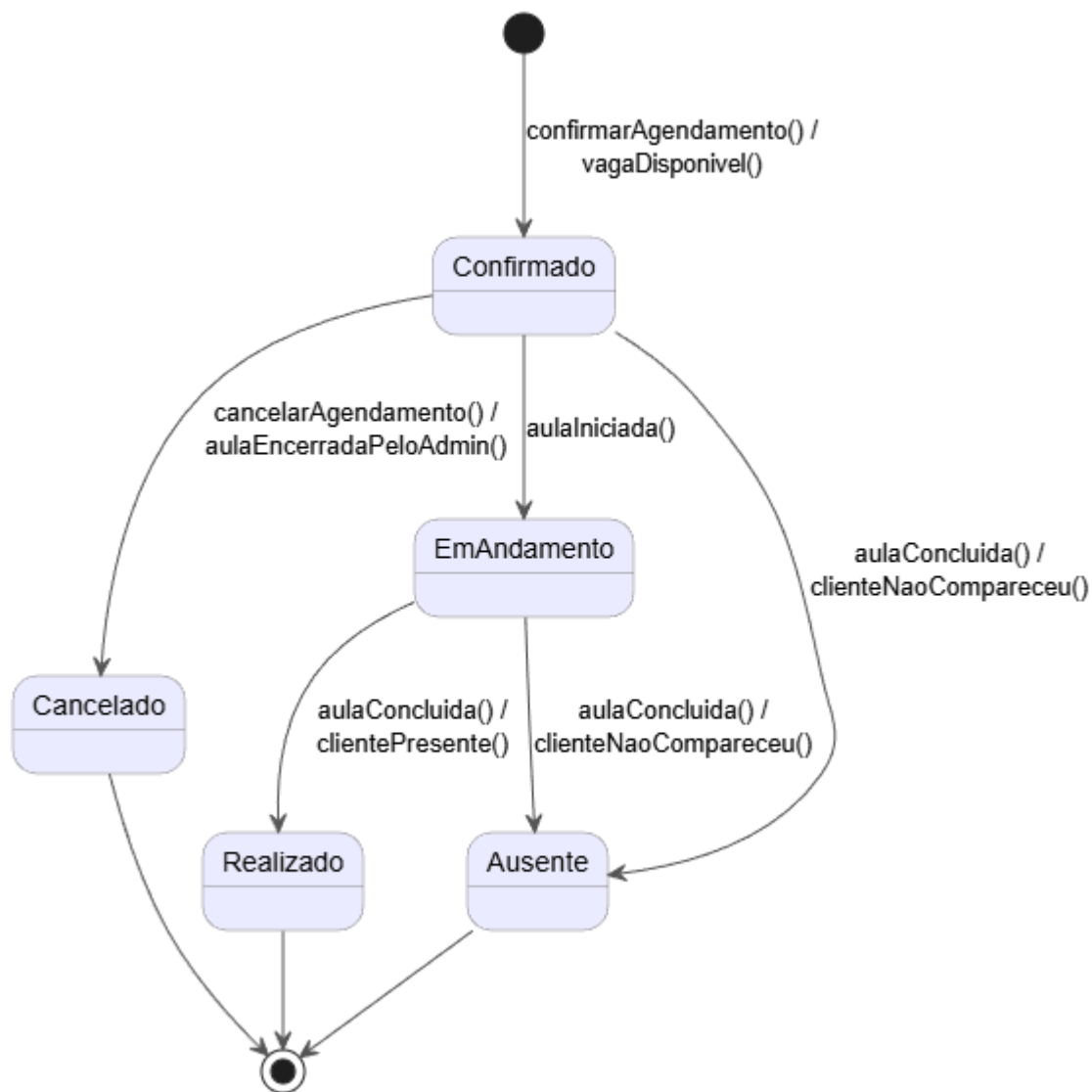


3.6 Diagramas de Estados

Nesta subseção são apresentados os Diagramas de Estados para as entidades centrais do sistema FitCore. Os diagramas modelam o ciclo de vida de Matrícula, Pagamento e Agendamento, evidenciando os estados possíveis de cada entidade e as transições disparadas por eventos do sistema ou ações dos usuários.

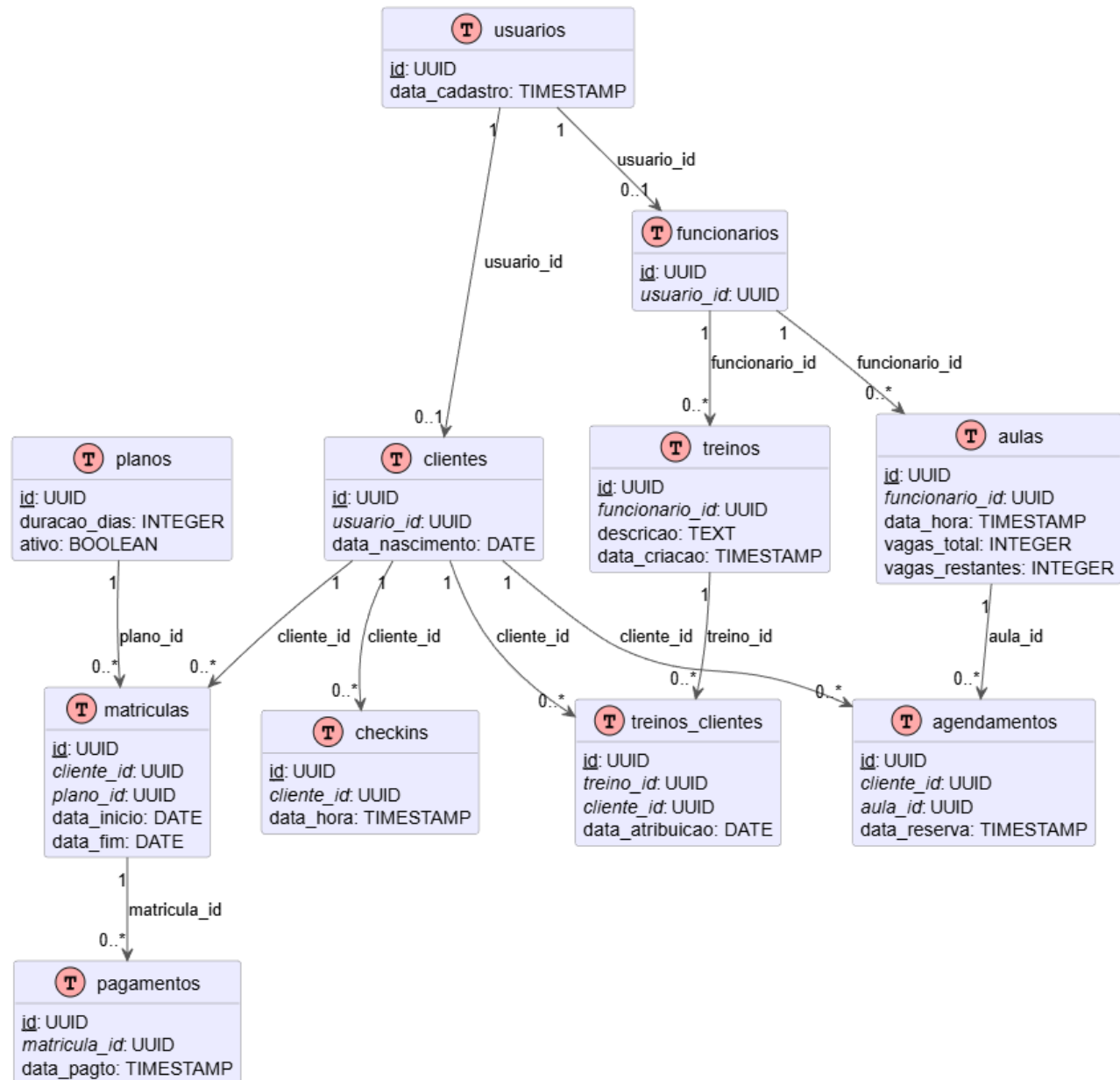






4. Modelos de Dados

Nesta seção é apresentado o modelo de dados do sistema FitCore. O diagrama entidade-relacionamento descreve as tabelas, atributos, chaves primárias e estrangeiras que compõem a base de dados relacional. Em seguida, é descrita a estratégia de mapeamento ORM adotada para conectar as entidades de domínio ao banco PostgreSQL.



Estratégia de Mapeamento:

Usuario (abstrata) → tabela: usuarios

Estratégia: Single Table Inheritance

Observação: Discriminador pela coluna tipo (cliente, funcionario, admin)

Cliente → tabela: clientes

Estratégia: Joined Table

Observação: Chave estrangeira para usuarios.id

Funcionario → tabela: funcionarios

Estratégia: Joined Table

Observação: Chave estrangeira para usuarios.id

Matricula → tabela: matriculas

Estratégia: One Table per Class

Observação: Status mapeado como Enum (pendente, ativa, suspensa, vencida, cancelada)

Pagamento → tabela: pagamentos

Estratégia: One Table per Class

Observação: Campo stripe_intent_id utilizado para reconciliação com o gateway externo

Checkin → tabela: checkins

Estratégia: One Table per Class

Observação: Registro imutável após inserção

Aula → tabela: aulas

Estratégia: One Table per Class

Observação: Campo vagas_restantes gerenciado via Redis para controle de concorrência

Agendamento → tabela: agendamentos

Estratégia: One Table per Class

Observação: Constraint única em (cliente_id, aula_id) para evitar duplicatas

Treino → tabela: treinos

Estratégia: One Table per Class

Observação: Nível de dificuldade mapeado como Enum (iniciante, intermediario, avancado)

TreinoCliente → tabela: treinos_clientes

Estratégia: Join Table explícita

Observação: Relacionamento N:N entre Treino e Cliente com campo adicional data_atribuicao